```
from google.colab import drive
drive.mount('/content/drive/')
```

```
Drive already mounted at /content/drive/; to attempt to forcibly remount, call drive.mount("/content/drive/", force_remount=True).
```

```
!pip install spectral
```

```
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public/simple/
Requirement already satisfied: spectral in /usr/local/lib/python3.9/dist-packages (0.23.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.9/dist-packages (from spectral) (1.22.4)
```

```
%tensorflow_version 2.x
import tensorflow as tf
import random
from tensorflow.keras import Sequential,layers
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import ModelCheckpoint
import statistics
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report, cohen_kappa_score
import tensorflow_probability as tfp
from operator import truediv
from tensorflow.compat.v1.distributions import Bernoulli
from plotly.offline import init_notebook_mode

import numpy as np
import matplotlib.pyplot as plt
import scipy.io as sio
import os
import spectral
import tensorflow as tf
from tensorflow.python.keras import backend as K

init_notebook_mode(connected=True)
%matplotlib inline
```

```
Colab only includes TensorFlow 2.x; %tensorflow_version has no effect.
```

```
# Global variables
im_width, im_height, im_depth, im_channel = 11,11,30, 1
mc_loss_weight = 5
```

Model Construction

[ ] ↳ *5 cells hidden*

Data Loading

[ ] ↳ *10 cells hidden*

Training

[ ] ↳ *4 cells hidden*

Tuning_5

[ ] ↳ *6 cells hidden*

Testing

```
def test_episode(test_patches_class,test_class_labels,test_C,test_K,i,f) :
  selected_classes = test_class_labels[i:f]   # [1, 2, 3, 4, 5, 6, 7, 8]
  support_labels = list(selected_classes)
  query_labels = []
  support_patches = []
  query_patches = []
  for x in selected_classes :
    y = test_class_labels.index(x)
```

```
    support_imgs = test_patches_class[y][:test_K,:,:,:,:]
    query_imgs = test_patches_class[y][test_K:,:,:,:,:]
    support_patches.extend(support_imgs)
    query_patches.extend(query_imgs)
    for i in range(query_imgs.shape[0]) :
      query_labels.append(x)
  temp1 = list(zip(query_patches, query_labels))
  random.shuffle(temp1)
  query_patches, query_labels = zip(*temp1)
  x = len(query_labels)
  query_patches = tf.convert_to_tensor(np.reshape(np.asarray(query_patches),(x,im_height,im_width,im_depth,1)),dtype=tf.float32)
  support_patches = tf.convert_to_tensor(np.reshape(np.asarray(support_patches),(test_C*test_K,im_height,im_width,im_depth,1)),dtype=tf.f
  return query_patches, support_patches, query_labels, support_labels,x
```

```
query_patches, support_patches, query_labels, support_labels,x = test_episode(test_patches_class,test_class_labels,3,5,0,3)
```

```
# Metrics to gather
test_loss = tf.metrics.Mean(name='test_loss')
test_acc = tf.metrics.Mean(name='test_accuracy')
def test_step(support, query, support_labels, query_labels, K, C, y):
    loss, mc_predictions, mean_accuracy, classwise_mean_acc, y = ProtoModel(support, query, support_labels, query_labels, K, C, y,n_times
    return loss, mc_predictions, mean_accuracy, classwise_mean_acc, y
```

```
for epoch in range(1): #1000
    test_loss.reset_states()
    test_acc.reset_states()
    tquery_patches1, tsupport_patches1, query_labels1, support_labels1, x1 = test_episode(test_patches_class,test_class_labels,3,5,0,3)
    loss1, mc_predictions1, mean_accuracy1, classwise_mean_acc1, y1 = test_step(tsupport_patches1, tquery_patches1,support_labels1, query
    print('OA1',mean_accuracy1)
# Class-wise Accuracy
    for i in range(tC) :
      print('class',i+1,classwise_mean_acc1[i])
    print('loss',loss1)
    tquery_patches2, tsupport_patches2, query_labels2, support_labels2, x2 = test_episode(test_patches_class,test_class_labels,3,5,3,6)
    loss2, mc_predictions2, mean_accuracy2, classwise_mean_acc2, y2 = test_step(tsupport_patches2, tquery_patches2,support_labels2, query
    print('OA2',mean_accuracy2)
# Class-wise Accuracy
    for i in range(tC) :
      print('class',i+4,classwise_mean_acc2[i])
    print('loss',loss2)
```

```
Loop Number :  9
AAAAAAAAAAAAAAAAAAAA
Dists :  tf.Tensor([643.871    317.05212   55.241127], shape=(3,), dtype=float32)
BBBBBBBBBBBBBBBBBBBB
New Distance (T applied):  tf.Tensor([585.3372   288.2292    50.219204], shape=(3,), dtype=float32)
CCCCCCCCCCCCCCCCCCCC
Softmax Predictions :  tf.Tensor([0. 0. 1.], shape=(3,), dtype=float32)
DDDDDDDDDDDDDDDDDDDD
Temperature Scaling Predictions :  tf.Tensor([0. 0. 1.], shape=(3,), dtype=float32)

OA2 tf.Tensor(0.9867987, shape=(), dtype=float32)
class 4 [1.0]
class 5 [0.98]
class 6 [1.0]
loss tf.Tensor(70.27353, shape=(), dtype=float32)
```

Overall Accuracy

```
mean_predictions1 =  tf.reduce_mean(mc_predictions1,axis=0)
mean_predictions2 =  tf.reduce_mean(mc_predictions2,axis=0)
overall_predictions = tf.concat([mean_predictions1,mean_predictions2],axis=0)
overall_true_labels = tf.concat([y1,y2],axis=0)
correct_pred = tf.cast(tf.equal(                                          # accuracy for the current pass
            tf.cast(tf.argmax(overall_predictions, axis=-1), tf.int32),
            tf.cast(tf.argmax(overall_true_labels,axis=-1), tf.int32)), tf.float32)
o_acc = tf.reduce_mean(correct_pred)
print("Overall accuracy:",o_acc)
```

```
Overall accuracy: tf.Tensor(0.82470787, shape=(), dtype=float32)
```

Confusion Matrix

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
mean_predictions1 =  tf.reduce_mean(mc_predictions1,axis=0)
cm_pred1 = tf.argmax(mean_predictions1, axis=-1)
mean_predictions2 =  tf.reduce_mean(mc_predictions2,axis=0)
cm_pred2 = tf.argmax(mean_predictions2, axis=-1) + 3
overall_predictions = tf.concat([cm_pred1,cm_pred2],axis=0)
cm_true1 = tf.argmax(y1,axis=-1)
cm_true2 = tf.argmax(y2,axis=-1) + 3
overall_true_labels = tf.concat([cm_true1,cm_true2],axis=0)
results = confusion_matrix(overall_true_labels,overall_predictions)
print ('Confusion Matrix :')
print(results)
print ('Report : ')
print (classification_report(overall_true_labels, overall_predictions))
```

```
Confusion Matrix :
[[ 15   0  26   0   0   0]
 [ 75 157   0   0   0   0]
 [  0   0  23   0   0   0]
 [  0   0   0  15   0   0]
 [  0   0   0   4 196   0]
 [  0   0   0   0   0  88]]
Report :
              precision    recall  f1-score   support

           0       0.17      0.37      0.23        41
           1       1.00      0.68      0.81       232
           2       0.47      1.00      0.64        23
           3       0.79      1.00      0.88        15
           4       1.00      0.98      0.99       200
           5       1.00      1.00      1.00        88

    accuracy                           0.82       599
   macro avg       0.74      0.84      0.76       599
weighted avg       0.92      0.82      0.85       599
```

Kappa Accuracy

```
import sklearn
sklearn.metrics.cohen_kappa_score(overall_true_labels, overall_predictions)
```

```
0.767340778527065
```

0s completed at 12:56 PM